Name : C.Srihan Reddy
Ht.no : 2303A51962
Batch 24

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Lab Objectives

• Use AI to assist in designing and implementing fundamental data structures in Python.
• Learn how to prompt AI for structure creation, optimization, and documentation.
• Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables.
• Enhance code quality with AI-generated comments and performance suggestions.

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:
class Stack:
pass

Expected Output:

• A functional stack implementation with all required methods and docstrings.

Code:

```
C: > Users > PC > Downloads > aiac > stack.py > ...
#generate a code tom demonstrate stack and its operations
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)
        print(f'Pushed {item} to stack.')

    def pop(self):
        if not self.is_empty():
            item = self.stack.pop()
            print(f'Popped {item} from stack.')
            return item
        else:
            print('Stack is empty. Cannot pop.')

    def peek(self):
        if not self.is_empty():
            item = self.stack[-1]
            print(f'Top item is {item}.')
            return item
        else:
            print('Stack is empty. No top item.')

    def is_empty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)
# Demonstrating stack operations
if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.peek()
    stack.pop()
    stack.peek()
    print(f'Stack size: {stack.size()}')
    stack.pop()
    stack.pop()
    stack.pop()  # Attempt to pop from an empty stack
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/stack.py
Pushed 10 to stack.
Pushed 20 to stack.
Pushed 30 to stack.
Top item is 30.
Popped 30 from stack.
Top item is 20.
Stack size: 2
Popped 20 from stack.
Popped 10 from stack.
Stack is empty. Cannot pop.
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:
class Queue:
pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:

```
C: > Users > PC > Downloads > aiac > queue.py > Queue > __init__
class Queue:
    def __init__(self):
        self.queue = []
    def enqueue(self, item):
        self.queue.append(item)
        print(f'Enqueued {item} to queue.')
    def dequeue(self):
        if not self.is_empty():
            item = self.queue.pop(0)
            print(f'Dequeued {item} from queue.')
            return item
        else:
            print('Queue is empty. Cannot dequeue.')
    def peek(self):
        if not self.is_empty():
            item = self.queue[0]
            print(f'Front item is {item}.')
            return item
        else:
            print('Queue is empty. No front item.')
    def is_empty(self):
        return len(self.queue) == 0
    def size(self):
        return len(self.queue)
# Demonstrating queue operations
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)
    queue.peek()
    queue.dequeue()
    queue.peek()
    print(f'Queue size: {queue.size()}')
    queue.dequeue()
    queue.dequeue()
    queue.dequeue()  # Attempt to dequeue from an empty queue
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/queue.py
Enqueued 10 to queue.
Enqueued 20 to queue.
Enqueued 30 to queue.
Front item is 10.
Dequeued 10 from queue.
Front item is 20.
Queue size: 2
Dequeued 20 from queue.
Dequeued 30 from queue.
Queue is empty. Cannot dequeue.
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

Code:

```
C: > Users > PC > Downloads > aiac > linkedList.py > ...
#implement a code for linked list and its operations
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            print(f'Appended {data} as head of the linked list.')
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
        print(f'Appended {data} to the linked list.')

    def prepend(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f'Prepended {data} to the linked list.')

    def delete_with_value(self, data):
        if not self.head:
            print('Linked list is empty. Cannot delete.')
            return
```

```python
        if self.head.data == data:
            self.head = self.head.next
            print(f'Deleted {data} from the linked list.')
            return
        current_node = self.head
        while current_node.next:
            if current_node.next.data == data:
                current_node.next = current_node.next.next
                print(f'Deleted {data} from the linked list.')
                return
            current_node = current_node.next
        print(f'{data} not found in the linked list.')

    def display(self):
        nodes = []
        current_node = self.head
        while current_node:
            nodes.append(str(current_node.data))
            current_node = current_node.next
        print(' -> '.join(nodes))
# Demonstrating linked list operations
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.append(10)
    linked_list.append(20)
    linked_list.append(30)
    linked_list.display()
    linked_list.prepend(5)
    linked_list.display()
    linked_list.delete_with_value(20)
    linked_list.display()
    linked_list.delete_with_value(40)  # Attempt to delete a non-existent value
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/linkedList.py
Appended 10 as head of the linked list.
Appended 20 to the linked list.
Appended 30 to the linked list.
10 -> 20 -> 30
Prepended 5 to the linked list.
5 -> 10 -> 20 -> 30
Deleted 20 from the linked list.
5 -> 10 -> 30
40 not found in the linked list.
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

• BST implementation with recursive insert and traversal methods.

Code:

C: > Users > PC > Downloads > aiac > bst.py > ...

```python
#create a python code to implement a BST with insert and in-order traversal methods.
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BST:
    def __init__(self):
        self.root = None
    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursively(self.root, key)
    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = Node(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = Node(key)
            else:
                self._insert_recursively(node.right, key)
    def in_order_traversal(self):
        return self._in_order_traversal_recursively(self.root)
    def _in_order_traversal_recursively(self, node):
        res = []
        if node:
            res = self._in_order_traversal_recursively(node.left)
            res.append(node.val)
```

```python
            res = res + self._in_order_traversal_recursively(node.right)
        return res
# Example usage:
if __name__ == "__main__":
    bst = BST()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("In-order Traversal:", bst.in_order_traversal())
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/bst.py
In-order Traversal: [2, 3, 4, 5, 6, 7, 8]
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Code:

C: > Users > PC > Downloads > aiac > hashtable.py > ...

```python
#generate a python code to implement a hash table with basic insert, search, anddelete methods.
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [None] * self.size
    def hash_function(self, key):
        return hash(key) % self.size
    def insert(self, key, value):
        index = self.hash_function(key)
        if self.table[index] is None:
            self.table[index] = [(key, value)]
        else:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    self.table[index][i] = (key, value)  # Update existing key
                    return
            self.table[index].append((key, value))  # Add new key-value pair
    def search(self, key):
        index = self.hash_function(key)
        if self.table[index] is not None:
            for k, v in self.table[index]:
                if k == key:
                    return v
        return None  # Key not found
    def delete(self, key):
        index = self.hash_function(key)
        if self.table[index] is not None:
            for i, (k, v) in enumerate(self.table[index]):
                if k == key:
                    del self.table[index][i]  # Remove the key-value pair
                    return True
        return False  # Key not found
# Example usage
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    print(hash_table.search("name"))  # Output: Alice
    print(hash_table.search("age"))  # Output: 30
    hash_table.delete("name")
    print(hash_table.search("name"))  # Output: None
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/hashtable.py
Alice
30
None
```

Task Description #6 – Graph Representation
Task: Use AI to implement a graph using an adjacency list.
Sample Input Code:
class Graph:
pass
Expected Output:
• Graph with methods to add vertices, add edges, and display connections.
Code:

C: > Users > PC > Downloads > aiac > graph.py > Graph > add_vertices

```python
#write a python code to implement a graph using an adjacency list
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append(v)
        self.graph[v].append(u)
    def add_vertices(self, vertices):
        for vertex in vertices:
            if vertex not in self.graph:
                self.graph[vertex] = []
    def display(self):
        for vertex in self.graph:
            print(f"{vertex}: {self.graph[vertex]}")
# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_edge(1, 2)
    g.add_edge(1, 3)
    g.add_edge(2, 4)
    g.add_vertices([5, 6])
    g.display()
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/graph.py
1: [2, 3]
2: [1, 4]
3: [1]
4: [2]
5: []
6: []
```

Task Description #7 – Priority Queue
Task: Use AI to implement a priority queue using Python's heapq module.
Sample Input Code:
class PriorityQueue:
pass
Expected Output:
• Implementation with enqueue (priority), dequeue (highest priority), and display methods.
Code:

C: > Users > PC > Downloads > aiac > priorityQueue.py > ...

```python
#generate a python code to implement a priority queue using Python's heapq module.
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))
    def dequeue(self):
        return heapq.heappop(self.elements)[1]
    def is_empty(self):
        return not self.elements
    def peek(self):
        return self.elements[0][1] if self.elements else None
    def size(self):
        return len(self.elements)
# Example usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("task1", priority=2)
    pq.enqueue("task2", priority=1)
    pq.enqueue("task3", priority=3)

    print("Next task:", pq.peek())  # Output: Next task: task2
    print("Dequeue:", pq.dequeue())  # Output: Dequeue: task2
    print("Next task:", pq.peek())  # Output: Next task: task1
    print("Queue size:", pq.size())  # Output: Queue size: 2
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/graph.py
1: [2, 3]
2: [1, 4]
3: [1]
4: [2]
5: []
6: []
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Code:

```
C: > Users > PC > Downloads > aiac > dequeue.py > ...
#generate a python code o implement a double-ended queue using collections.deque
from collections import deque
class DoubleEndedQueue:
    def __init__(self):
        self.deque = deque()
    def add_front(self, item):
        self.deque.appendleft(item)
    def add_rear(self, item):
        self.deque.append(item)
    def remove_front(self):
        if not self.is_empty():
            return self.deque.popleft()
        else:
            raise IndexError("Deque is empty")
    def remove_rear(self):
        if not self.is_empty():
            return self.deque.pop()
        else:
            raise IndexError("Deque is empty")
    def is_empty(self):
        return len(self.deque) == 0
    def size(self):
        return len(self.deque)
# Example usage
if __name__ == "__main__":
    deq = DoubleEndedQueue()
    deq.add_rear(1)
    deq.add_rear(2)
    deq.add_front(0)
    print("Deque after adding elements:", list(deq.deque))
    print("Removed from front:", deq.remove_front())
    print("Removed from rear:", deq.remove_rear())
    print("Deque after removing elements:", list(deq.deque))
    print("Is the deque empty?", deq.is_empty())
    print("Size of the deque:", deq.size())
```

Output:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/dequeue.py
Deque after adding elements: [0, 1, 2]
Removed from front: 0
Removed from rear: 2
Deque after removing elements: [1]
Is the deque empty? False
Size of the deque: 1
```

Task Description #9 Real-Time Application Challenge – Choose theRight Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System
that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

Feature	Data Structure
1. Attendance Tracking	Hash Table
2. Event Registration	Binary Search Tree (BST)
3. Library Borrowing	Linked List
4. Bus Scheduling	Graph
5. Cafeteria Queue	Queue

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings

Code 1:

C: > Users > PC > Downloads > aiac > student_Attendance_hashtable.py > ...

```python
# generate a python code for maintaining a daily log of students entering/exiting the campus using a hashtable (dictionary) to store the log data.
import datetime
class CampusLog:
    def __init__(self):
        self.log = {}

    def log_entry(self, student_id):
        timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        self.log[student_id] = {"entry_time": timestamp, "exit_time": None}
        print(f"Student {student_id} entered the campus at {timestamp}.")

    def log_exit(self, student_id):
        if student_id in self.log and self.log[student_id]["exit_time"] is None:
            timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            self.log[student_id]["exit_time"] = timestamp
            print(f"Student {student_id} exited the campus at {timestamp}.")
        else:
            print(f"Student {student_id} has not entered the campus or has already exited.")

    def display_log(self):
        for student_id, times in self.log.items():
            entry_time = times["entry_time"]
            exit_time = times["exit_time"] if times["exit_time"] else "Still on campus"
            print(f"Student ID: {student_id}, Entry Time: {entry_time}, Exit Time: {exit_time}")
# Example usage
if __name__ == "__main__":
    campus_log = CampusLog()
    campus_log.log_entry("S001")
    campus_log.log_entry("S002")
    campus_log.log_exit("S001")
    campus_log.display_log()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/student_Attendance_hashtable.py
Student S001 entered the campus at 2026-03-13 21:31:59.
Student S002 entered the campus at 2026-03-13 21:31:59.
Student S001 exited the campus at 2026-03-13 21:31:59.
Student ID: S001, Entry Time: 2026-03-13 21:31:59, Exit Time: 2026-03-13 21:31:59
Student ID: S002, Entry Time: 2026-03-13 21:31:59, Exit Time: Still on campus
```

Code 2:

C: > Users > PC > Downloads > aiac > CafeteriaOrderQueue.py > ...

```python
# write a python program to manage a cafeteria order queue for students using a Queue data structure.
from collections import deque
class CafeteriaQueue:
    """
    A class to manage student cafeteria orders using Queue data structure.
    """
    def __init__(self):
        # Initializing an empty deque to store orders
        self.queue = deque()
    def place_order(self, student_name, order_item):
        """Adds a student's order to the end of the queue."""
        self.queue.append({"name": student_name, "item": order_item})
        print(f"Order placed: {student_name} wants {order_item}.")
    def serve_order(self):
        """Removes and returns the order at the front of the queue."""
        if not self.is_empty():
            order = self.queue.popleft()
            print(f"Serving {order['name']}'s {order['item']}. Enjoy!")
            return order
        print("The queue is empty! No orders to serve.")
        return None
    def is_empty(self):
        """Checks if there are any orders remaining."""
        return len(self.queue) == 0
    def view_queue(self):
        """Displays all pending orders in the queue."""
        if self.is_empty():
            print("Queue is currently empty.")
        else:
            print("\n--- Current Cafeteria Queue ---")
            for idx, order in enumerate(self.queue, 1):
                print(f"{idx}. {order['name']} - {order['item']}")
        print("------------------------------\n")
# --- Example Usage ---
if __name__ == "__main__":
    campus_cafe = CafeteriaQueue()
    # Students arriving and ordering
    campus_cafe.place_order("Alice", "Chicken Wrap")
    campus_cafe.place_order("Bob", "Veggie Burger")
    campus_cafe.place_order("Charlie", "Iced Coffee")
    # Show the current line
    campus_cafe.view_queue()
    # Serving the first two students
    campus_cafe.serve_order()
    campus_cafe.serve_order()
    # Final status
    campus_cafe.view_queue()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/CafeteriaOrderQueue.py
Order placed: Alice wants Chicken Wrap.
Order placed: Bob wants Veggie Burger.
Order placed: Charlie wants Iced Coffee.

--- Current Cafeteria Queue ---
1. Alice - Chicken Wrap
2. Bob - Veggie Burger
3. Charlie - Iced Coffee
------------------------------

Serving Alice's Chicken Wrap. Enjoy!
Serving Bob's Veggie Burger. Enjoy!

--- Current Cafeteria Queue ---
1. Charlie - Iced Coffee
------------------------------
```

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure
1. Patient Check-In	Queue
2. Emergency Handling	Priority Queue
3. Medical Records	Hash Table
4. Appointment Scheduling	Binary Search Tree (BST)
5. Room Navigation	Graph

Code 1:

C: > Users > PC > Downloads > aiac > emergencyPriorityQueue.py > EmergencyRoom > __init__

```python
# write a python program to manage an emergency room patient queue based on triage priority using a Priority Queue data struct
import heapq
class EmergencyRoom:
    """
    A system to manage emergency patients based on triage priority.
    Priority 1 is the most urgent (Critical), while higher numbers are less urgent.
    """
    def __init__(self):
        # The heap stores tuples: (priority_level, patient_name)
        self.patient_heap = []

    def admit_patient(self, name, priority):
        """
        Adds a patient to the emergency queue.
        :param name: Name of the patient.
        :param priority: Integer (1 = Critical, 2 = Serious, 3 = Stable).
        """
        heapq.heappush(self.patient_heap, (priority, name))
        print(f"Admitted: {name} (Priority Level: {priority})")
    def treat_next_patient(self):
        """Removes and returns the patient with the highest priority."""
        if not self.patient_heap:
            print("No patients in the waiting area.")
            return None
        priority, name = heapq.heappop(self.patient_heap)
        status = {1: "CRITICAL", 2: "SERIOUS", 3: "STABLE"}.get(priority, "ROUTINE")
        print(f"\n>>> Treating Patient: {name} [Status: {status}]")
        return name
    def show_waiting_list(self):
        """Displays all patients currently waiting, sorted by priority."""
        if not self.patient_heap:
            print("The waiting room is empty.")
        else:
            # We sort a copy to show the user without affecting the heap structure
            sorted_list = sorted(self.patient_heap)
            print("\n--- Current Emergency Waiting List ---")
            for p, name in sorted_list:
                print(f"Priority {p}: {name}")
            print("------------------------------------")
# Example usage
if __name__ == "__main__":
    er = EmergencyRoom()
    # Patients arrive in different orders
    er.admit_patient("John Doe", 3)      # Stable
    er.admit_patient("Jane Smith", 1)    # Critical (Arrived second, but served first)
    er.admit_patient("Baby Sam", 2)      # Serious
    # View the priority list
    er.show_waiting_list()
    # Treat the most urgent cases first
    er.treat_next_patient()  # Should be Jane Smith (Priority 1)
    er.treat_next_patient()  # Should be Baby Sam (Priority 2)
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/emergencyPriorityQueue.py
Admitted: John Doe (Priority Level: 3)
Admitted: Jane Smith (Priority Level: 1)
Admitted: Baby Sam (Priority Level: 2)

--- Current Emergency Waiting List ---
Priority 1: Jane Smith
Priority 2: Baby Sam
Priority 3: John Doe
------------------------------------

>>> Treating Patient: Jane Smith [Status: CRITICAL]

>>> Treating Patient: Baby Sam [Status: SERIOUS]
PS E:\Microsoft VS Code>
```

Code 2:

```
C: > Users > PC > Downloads > aiac > hospitalRoomNavigation.py > HospitalGraph > display_map
from collections import defaultdict
class HospitalGraph:
    def __init__(self):
        # Using a dictionary to store the adjacency list representation of the graph
        self.graph = defaultdict(list)
    def add_room(self, room):
        """Add a room to the hospital graph."""
        if room not in self.graph:
            self.graph[room] = []
    def add_connection(self, room1, room2):
        """Add a bidirectional connection between two rooms."""
        self.graph[room1].append(room2)
        self.graph[room2].append(room1)
    def get_connections(self, room):
        """Get all rooms connected to the specified room."""
        return self.graph[room]
    def display_map(self):
        """Display the entire hospital map."""
        print("\n--- Hospital Map ---")
        for room, connections in self.graph.items():
            print(f"{room}: {', '.join(connections)}")
# Example usage
if __name__ == "__main__":
    hospital = HospitalGraph()
    # Add rooms
    hospital.add_room("Ward A")
    hospital.add_room("Ward B")
    hospital.add_room("Ward C")
    hospital.add_room("Emergency Room")
    hospital.add_room("ICU")
    # Add connections
    hospital.add_connection("Ward A", "Ward B")
    hospital.add_connection("Ward B", "Ward C")
    hospital.add_connection("Ward A", "Emergency Room")
    hospital.add_connection("ICU", "Ward C")
    # Display the map
    hospital.display_map()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/hospitalRoomNavigation.py

--- Hospital Map ---
Ward A: Ward B, Emergency Room
Ward B: Ward A, Ward C
Ward C: Ward B, ICU
Emergency Room: Ward A
ICU: Ward C
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure
1. Traffic Signal Queue	**Queue**
2. Emergency Priority	**Priority Queue**
3. Vehicle Lookup	**Hash Table**

Feature	Data Structure
4. Road Network Mapping	Graph
5. Parking Availability	Hash Table

Code 1:

C: > Users > PC > Downloads > aiac > trafficSignal.py > ...

```python
# write a python program to display Traffic Signal Queue – Vehicles waiting at signals
import time
import random
class TrafficSignal:
    def __init__(self):
        self.vehicles_queue = []
    def add_vehicle(self, vehicle):
        self.vehicles_queue.append(vehicle)
    def display_queue(self):
        print("Vehicles waiting at the signal:")
        for vehicle in self.vehicles_queue:
            print(vehicle)
    def change_signal(self):
        if self.vehicles_queue:
            print("Signal is green. Vehicles can go.")
            self.vehicles_queue.clear()
        else:
            print("Signal is red. No vehicles waiting.")
def main():
    traffic_signal = TrafficSignal()
    vehicle_types = ['Car', 'Bus', 'Motorcycle', 'Truck']
    for _ in range(10):
        vehicle = random.choice(vehicle_types)
        traffic_signal.add_vehicle(vehicle)
        time.sleep(1)  # Simulate time delay for vehicles arriving
    traffic_signal.display_queue()
    traffic_signal.change_signal()
if __name__ == "__main__":
    main()
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/trafficSignal.py
Vehicles waiting at the signal:
Motorcycle
Truck
Truck
Truck
Car
Truck
Bus
Bus
Bus
Bus
Signal is green. Vehicles can go.
```

Code 2:

C: > Users > PC > Downloads > aiac > vechileRegistry.py > ...

```python
#write a python program that simulates a vehicle registration system using a hash table (dictionary) for instant lookups.
class VehicleRegistry:
    # A system to manage vehicle registrations using a Hash Table
    def __init__(self):
        # The dictionary acts as our Hash Table
        # Key: License Plate (String), Value: Dictionary of Vehicle Details
        self.registry = {}
    def register_vehicle(self, plate_number, owner, model, color):
        """Adds a new vehicle to the system."""
        self.registry[plate_number.upper()] = {
            "owner": owner,
            "model": model,
            "color": color
        }
        print(f"Registered: [{plate_number.upper()}] for {owner}")
    def lookup_vehicle(self, plate_number):
        """Retrieves vehicle details instantly using the plate number."""
        plate = plate_number.upper()
        if plate in self.registry:
            details = self.registry[plate]
            print(f"\n--- Vehicle Details for {plate} ---")
            print(f"Owner: {details['owner']}")
            print(f"Vehicle: {details['color']} {details['model']}")
            print("----------------------------------")
            return details
        else:
            print(f"\nError: Vehicle with plate {plate} not found.")
            return None
    def remove_vehicle(self, plate_number):
        """Removes a vehicle record from the system."""
```

```python
        if plate_number.upper() in self.registry:
            del self.registry[plate_number.upper()]
            print(f"Record for {plate_number.upper()} has been deleted.")
        else:
            print("Plate not found. Nothing to delete.")

# Example usage
if __name__ == "__main__":
    city_dmv = VehicleRegistry()
    # Registering some vehicles
    city_dmv.register_vehicle("GMNI-2026", "Dr. Sarah Smith", "Tesla Model S", "White")
    city_dmv.register_vehicle("CITY-4455", "Marcus Vane", "Ford F-150", "Blue")
    city_dmv.register_vehicle("FAST-9999", "Elena Rodriguez", "Porsche 911", "Red")
    # Instant Lookup Example
    city_dmv.lookup_vehicle("GMNI-2026")
    # Searching for a non-existent plate
    city_dmv.lookup_vehicle("LOST-0000")
    # Removing a record
    city_dmv.remove_vehicle("CITY-4455")
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/vechileRegistry.py
Registered: [GMNI-2026] for Dr. Sarah Smith
Registered: [CITY-4455] for Marcus Vane
Registered: [FAST-9999] for Elena Rodriguez

--- Vehicle Details for GMNI-2026 ---
Owner: Dr. Sarah Smith
Vehicle: White Tesla Model S
---------------------------------

Error: Vehicle with plate LOST-0000 not found.
Record for CITY-4455 has been deleted.
```

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure
1. Shopping Cart	**Linked List**
2. Order Processing	**Queue**
3. Top-Selling Tracker	**Priority Queue**

Feature	Data Structure
4. Product Search	Hash Table
5. Delivery Routes	Graph

Code 1:

C: > Users > PC > Downloads > aiac > TopSellingTracker.py > TopSellingTracker > __init__

```python
#write a pythn program that tracks the top-selling items in a store using a Max-Priority Queue
import heapq
class TopSellingTracker:
    """
    A system to track and rank products based on sales count using
    a Max-Priority Queue (implemented via a min-heap with negative values).
    """
    def __init__(self):
        # We store items as [-sales_count, product_name] to simulate a Max-Heap
        self.heap = []
    def record_sale(self, product_name, sales_count):
        """
        Adds or updates a product's sales data.
        :param product_name: The name of the product.
        :param sales_count: The total number of units sold.
        """
        # We push negative sales_count so the highest number is at the 'top'
        heapq.heappush(self.heap, (-sales_count, product_name))
        print(f"Recorded: {product_name} with {sales_count} sales.")
    def get_top_seller(self):
        """Retrieves the current #1 trending product."""
        if not self.heap:
            return "No sales recorded yet."
        neg_sales, name = self.heap[0]
        return f"Current Top Seller: {name} ({abs(neg_sales)} units sold)"
    def display_rankings(self):
        """displays all products ranked from most sales to least."""
        if not self.heap:
            print("No rankings available.")
            return
        # Create a sorted copy for display without destroying the heap
        sorted_ranks = sorted(self.heap)
        print("\n--- Smart Shop: Top Sales Rankings ---")
        for rank, (neg_sales, name) in enumerate(sorted_ranks, 1):
            print(f"Rank {rank}: {name} | Sales: {abs(neg_sales)}")
        print("------------------------------------\n")
# --- Example Usage ---
if __name__ == "__main__":
    tracker = TopSellingTracker()
    # Recording sales for various items
    tracker.record_sale("Wireless Earbuds", 1250)
    tracker.record_sale("Ergonomic Chair", 450)
    tracker.record_sale("Gaming Laptop", 2100)
    tracker.record_sale("Mechanical Keyboard", 890)
    # Show the current rankings
    tracker.display_rankings()
    # Check the #1 item
    print(tracker.get_top_seller())
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/TopSellingTracker.py
Recorded: Wireless Earbuds with 1250 sales.
Recorded: Ergonomic Chair with 450 sales.
Recorded: Gaming Laptop with 2100 sales.
Recorded: Mechanical Keyboard with 890 sales.

--- Smart Shop: Top Sales Rankings ---
Rank 1: Gaming Laptop | Sales: 2100
Rank 2: Wireless Earbuds | Sales: 1250
Rank 3: Mechanical Keyboard | Sales: 890
Rank 4: Ergonomic Chair | Sales: 450
------------------------------------

Current Top Seller: Gaming Laptop (2100 units sold)
```

Code 2:

```
C: > Users > PC > Downloads > aiac > shopping_cart.py > ...
#generate a python program that prints Shopping Cart Management – Add and remove products dynamically using linked list data structure.
class Node:
    def __init__(self, product_name, price):
        self.product_name = product_name
        self.price = price
        self.next = None
class ShoppingCart:
    def __init__(self):
        self.head = None
    def add_product(self, product_name, price):
        new_node = Node(product_name, price)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def remove_product(self, product_name):
        current_node = self.head
        previous_node = None
        while current_node:
            if current_node.product_name == product_name:
                if previous_node:
                    previous_node.next = current_node.next
                else:
                    self.head = current_node.next
                return
            previous_node = current_node
            current_node = current_node.next
    def display_cart(self):
        current_node = self.head
        while current_node:
            print(f"Product: {current_node.product_name}, Price: {current_node.price}")
            current_node = current_node.next
# Example usage
if __name__ == "__main__":
    cart = ShoppingCart()
    cart.add_product("Laptop", 999.99)
    cart.add_product("Smartphone", 499.99)
    cart.add_product("Headphones", 199.99)
    print("Shopping Cart:")
    cart.display_cart()
    print("\nRemoving 'Smartphone' from the cart...")
    cart.remove_product("Smartphone")
    print("\nUpdated Shopping Cart:")
    cart.display_cart()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/shopping_cart.py
Shopping Cart:
Product: Laptop, Price: 999.99
Product: Smartphone, Price: 499.99
Product: Headphones, Price: 199.99

Removing 'Smartphone' from the cart...

Updated Shopping Cart:
Product: Laptop, Price: 999.99
Product: Headphones, Price: 199.99
```

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure
1. Shipment Processing	Queue
2. Urgent Shipments	Priority Queue
3. Product Lookup	Hash Table
4. Warehouse Network	Graph
5. Inventory Operations	Linked List

Code 1:

C: > Users > PC > Downloads > aiac > urgentShipment.py > LogisticsPrioritySystem > add_shipment

```python
#write a python program that simulates a logistics priority system for a warehouse
import heapq
class LogisticsPrioritySystem:
    """
    Manages warehouse shipments based on priority levels.
    Priority 1 = Urgent/Express, Priority 2 = Standard, Priority 3 = Bulk/Economy.
    """
    def __init__(self):
        # Min-heap stores tuples: (priority_level, shipment_id, description)
        self.shipment_heap = []

    def add_shipment(self, shipment_id, description, priority):
        """
        Adds a shipment to the processing heap.
        :param shipment_id: Unique string ID for the package.
        :param description: Brief details of the contents.
        :param priority: Integer (1=Highest, 3=Lowest).
        """
        # We include shipment_id in the tuple to handle cases with same priority
        heapq.heappush(self.shipment_heap, (priority, shipment_id, description))
        print(f"Added to System: {shipment_id} | Priority: {priority} ({description})")

    def process_next_shipment(self):
        """Extracts and processes the shipment with the highest priority (lowest number)."""
        if not self.shipment_heap:
            print("No shipments remaining in the warehouse.")
            return None

        priority, s_id, desc = heapq.heappop(self.shipment_heap)
        priority_map = {1: "URGENT", 2: "STANDARD", 3: "ECONOMY"}
        print(f"\n[PROCESSING] ID: {s_id} | Type: {priority_map.get(priority)} | Item: {desc}")
        return s_id
    def view_pending_shipments(self):
        """Displays all shipments currently waiting, sorted by priority."""
        if not self.shipment_heap:
            print("Warehouse is empty.")
        else:
            # Sort a copy to show correctly without destroying heap properties
            current_queue = sorted(self.shipment_heap)
            print("\n--- Current Processing Queue ---")
            for p, s_id, desc in current_queue:
                print(f"Priority {p}: {s_id} ({desc})")
            print("------------------------------")
# Example usage
if __name__ == "__main__":
    logistics = LogisticsPrioritySystem()
    # 1. Standard shipment arrives
    logistics.add_shipment("SHIP-101", "Office Supplies", 2)
    # 2. Bulk/Economy shipment arrives
    logistics.add_shipment("SHIP-102", "Construction Sand", 3)
    # 3. CRITICAL Urgent shipment arrives (must be processed first!)
    logistics.add_shipment("SHIP-999", "Medical Vaccines", 1)
    # View the queue (Vaccines should be at the top)
    logistics.view_pending_shipments()
    # Process shipments
    logistics.process_next_shipment() # Processes SHIP-999
    logistics.process_next_shipment() # Processes SHIP-101
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/urgentShipment.py
Added to System: SHIP-101 | Priority: 2 (Office Supplies)
Added to System: SHIP-102 | Priority: 3 (Construction Sand)
Added to System: SHIP-999 | Priority: 1 (Medical Vaccines)

--- Current Processing Queue ---
Priority 1: SHIP-999 (Medical Vaccines)
Priority 2: SHIP-101 (Office Supplies)
Priority 3: SHIP-102 (Construction Sand)
------------------------------

[PROCESSING] ID: SHIP-999 | Type: URGENT | Item: Medical Vaccines

[PROCESSING] ID: SHIP-101 | Type: STANDARD | Item: Office Supplies
```

Code 2:

C: > Users > PC > Downloads > aiac > wareHouse.py > ...

```python
#write a python program to display Warehouse Network Connectivity
class WarehouseNetwork:
    def __init__(self):
        self.network = {}

    def add_warehouse(self, warehouse_name):
        if warehouse_name not in self.network:
            self.network[warehouse_name] = []

    def connect_warehouses(self, warehouse1, warehouse2):
        if warehouse1 in self.network and warehouse2 in self.network:
            self.network[warehouse1].append(warehouse2)
            self.network[warehouse2].append(warehouse1)

    def display_network(self):
        for warehouse, connections in self.network.items():
            print(f"{warehouse} is connected to: {', '.join(connections)}")
# Example usage
if __name__ == "__main__":
    warehouse_network = WarehouseNetwork()
    warehouse_network.add_warehouse("Warehouse A")
    warehouse_network.add_warehouse("Warehouse B")
    warehouse_network.add_warehouse("Warehouse C")
    warehouse_network.connect_warehouses("Warehouse A", "Warehouse B")
    warehouse_network.connect_warehouses("Warehouse B", "Warehouse C")
    warehouse_network.display_network()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/wareHouse.py
Warehouse A is connected to: Warehouse B
Warehouse B is connected to: Warehouse A, Warehouse C
Warehouse C is connected to: Warehouse B
```

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.
2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure
1. Token System	Queue
2. Loan Priority	Priority Queue
3. Account Lookup	Hash Table
4. Transaction History	Linked List
5. ATM Network	Graph

Code 1:

C: > Users > PC > Downloads > aiac > bankRegistry.py > BankRegistry > deposit

```python
# wrute a python program to display a bank registry system using a hash table to store account information and provide instant lookup and update capabilities.
class BankRegistry:
    """
    A system to manage bank account records using a Hash Table.
    Provides O(1) average time complexity for lookups and updates.
    """
    def __init__(self):
        # Dictionary acts as the Hash Table: Key=AccountNum, Value=Details
        self.accounts = {}

    def create_account(self, account_num, name, balance):
        """Adds a new account to the registry."""
        if account_num in self.accounts:
            print(f"Error: Account {account_num} already exists.")
        else:
            self.accounts[account_num] = {
                "name": name,
                "balance": balance,
                "status": "Active"
            }
            print(f"Account created successfully for {name}.")

    def get_account_details(self, account_num):
        """Retrieves account information instantly using the account number."""
        account = self.accounts.get(account_num)
        if account:
            print(f"\n--- Account Statement: {account_num} ---")
            print(f"Holder: {account['name']}")
            print(f"Balance: ${account['balance']:,.2f}")
            print(f"Status: {account['status']}")
            print("----------------------------------------")
            return account
        else:
            print(f"Error: Account number {account_num} not found.")
            return None

    def deposit(self, account_num, amount):
        """Updates the balance of an existing account."""
        if account_num in self.accounts:
            self.accounts[account_num]['balance'] += amount
            print(f"Deposited ${amount}. New balance: ${self.accounts[account_num]['balance']:,.2f}")
        else:
            print("Transaction failed: Account not found.")
# Example usage
if __name__ == "__main__":
    bank_system = BankRegistry()
    # Onboarding customers
    bank_system.create_account("AC-1001", "Alice Smith", 5000.00)
    bank_system.create_account("AC-1002", "Bob Jones", 1250.75)
    bank_system.create_account("AC-1003", "Charlie Day", 10.00)
    # Instant Lookup
    bank_system.get_account_details("AC-1001")
    # Transaction (Update)
    bank_system.deposit("AC-1001", 1500.00)
    # Checking updated record
    bank_system.get_account_details("AC-1001")
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/bankRegistry.py
Account created successfully for Alice Smith.
Account created successfully for Bob Jones.
Account created successfully for Charlie Day.

--- Account Statement: AC-1001 ---
Holder: Alice Smith
Balance: $5,000.00
Status: Active
----------------------------------------
Deposited $1500.0. New balance: $6,500.00

--- Account Statement: AC-1001 ---
Holder: Alice Smith
Balance: $6,500.00
Status: Active
----------------------------------------
```

Code 2:

```
C: > Users > PC > Downloads > aiac > ATM_Network.py > ...
#write a python program to display ATM Network Connectivity - Represent connections between ATMs across cities
class ATMNetwork:
    def __init__(self):
        self.network = {}

    def add_atm(self, atm_name):
        if atm_name not in self.network:
            self.network[atm_name] = []

    def connect_atms(self, atm1, atm2):
        if atm1 in self.network and atm2 in self.network:
            self.network[atm1].append(atm2)
            self.network[atm2].append(atm1)

    def display_network(self):
        for atm, connections in self.network.items():
            print(f"{atm} is connected to: {', '.join(connections)}")
# Example usage
if __name__ == "__main__":
    atm_network = ATMNetwork()
    atm_network.add_atm("ATM 1 - New York")
    atm_network.add_atm("ATM 2 - Los Angeles")
    atm_network.add_atm("ATM 3 - Chicago")
    atm_network.connect_atms("ATM 1 - New York", "ATM 2 - Los Angeles")
    atm_network.connect_atms("ATM 2 - Los Angeles", "ATM 3 - Chicago")
    atm_network.display_network()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/ATM_Network.py
ATM 1 - New York is connected to: ATM 2 - Los Angeles
ATM 2 - Los Angeles is connected to: ATM 1 - New York, ATM 3 - Chicago
ATM 3 - Chicago is connected to: ATM 2 - Los Angeles
PS E:\Microsoft VS Code>
```

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from the given list.
• Justify your selection in 2–3 sentences per feature.
• Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

• Feature → Data Structure → Justification table.
• Functional Python program with proper documentation.

Feature	**Data Structure**
1. Login Authentication	**Hash Table**
2. Submission Queue	**Queue**
3. Top Performer Ranking	**Priority Queue**
4. Course Prerequisites	**Graph**
5. Undo Feature	**Stack**

Code 1:

C: > Users > PC > Downloads > aiac > editor_undo_system.py > EditorUndoSystem > write

```python
#write a python program that simulates a simple text editor with an Undo feature using a Stack data structure.
class EditorUndoSystem:
    """
    A system to manage document edits with an Undo capability
    using a Stack (LIFO) data structure.
    """
    def __init__(self):
        # The history stack stores previous versions of the text
        self.history = []
        self.current_text = ""

    def write(self, new_text):
        """
        Appends text to the current document and saves the previous state.
        :param new_text: String to be added to the document.
        """
        # Save current state to history before changing it
        self.history.append(self.current_text)
        self.current_text += new_text
        print(f"Typed: '{new_text}'")
        self.display_editor()

    def undo(self):
        """Reverts the text to the last saved state in the stack."""
        if not self.history:
            print("\n[Undo] Nothing left to undo!")
            return

        # Pop the last state from the stack
        self.current_text = self.history.pop()
        print("\n[Undo] Reverted last action.")
        self.display_editor()

    def display_editor(self):
        """Shows the current content of the editor."""
        print(f"Current Content: \"{self.current_text}\"")
        print("-" * 30)

# --- LMS Editor Simulation ---
if __name__ == "__main__":
    my_editor = EditorUndoSystem()

    # Student starts writing an essay
    my_editor.write("Hello ")
    my_editor.write("World! ")
    my_editor.write("This is a Python task.")

    # Oops, made a mistake! Let's undo
    my_editor.undo()

    # Let's undo again
    my_editor.undo()

    # Continue writing
    my_editor.write("Data structures are cool.")
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/editor_undo_system.py
Typed: 'Hello '
Current Content: "Hello "
------------------------------
Typed: 'World! '
Current Content: "Hello World! "
------------------------------
Typed: 'This is a Python task.'
Current Content: "Hello World! This is a Python task."
------------------------------

[Undo] Reverted last action.
Current Content: "Hello World! "
------------------------------

[Undo] Reverted last action.
Current Content: "Hello "
------------------------------
Typed: 'Data structures are cool.'
Current Content: "Hello Data structures are cool."
------------------------------
```

Code 2:

```
C: > Users > PC > Downloads > aiac > assignmentSubmission.py > ...
#write a python program to display Assignment Submission Queue – Submissions processed in order of upload
import heapq
class AssignmentSubmissionQueue:
    def __init__(self):
        self.submission_queue = []
        self.counter = 0  # To maintain the order of submissions

    def submit_assignment(self, student_name, assignment_name):
        # Use a tuple (counter, student_name, assignment_name) to maintain order
        heapq.heappush(self.submission_queue, (self.counter, student_name, assignment_name))
        self.counter += 1
        print(f"Assignment '{assignment_name}' submitted by {student_name}.")

    def process_submission(self):
        if self.submission_queue:
            submission = heapq.heappop(self.submission_queue)
            print(f"Processing submission: {submission[1]} - {submission[2]}")
        else:
            print("No submissions to process.")
# Example usage
if __name__ == "__main__":
    submission_queue = AssignmentSubmissionQueue()
    submission_queue.submit_assignment("Alice", "Math Homework")
    submission_queue.submit_assignment("Bob", "Science Project")
    submission_queue.submit_assignment("Charlie", "History Essay")

    submission_queue.process_submission()  # Should process Alice's submission first
    submission_queue.process_submission()  # Should process Bob's submission next
    submission_queue.process_submission()  # Should process Charlie's submission last
    submission_queue.process_submission()  # No more submissions to process

```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/assignmentSubmission.py
Assignment 'Math Homework' submitted by Alice.
Assignment 'Science Project' submitted by Bob.
Assignment 'History Essay' submitted by Charlie.
Processing submission: Alice - Math Homework
Processing submission: Bob - Science Project
Processing submission: Charlie - History Essay
No submissions to process.
```

Task Description #16: Smart Airport Management System – Data

Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

• Choose suitable data structures from the provided list.
• Provide 2–3 sentence justification per feature.
• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.
• Working Python implementation.

Feature	**Data Structure**
1. Passenger Check-In	**Queue**
2. Emergency Landing	**Priority Queue**

Feature	Data Structure
3. Flight Lookup	Hash Table
4. Air Route Connectivity	Graph
5. Cabin Storage	Stack

Code 1:

C: > Users > PC > Downloads > aiac > airTrafficControl.py > ...

```python
#write a python program that simulates an air traffic control system managing aircraft landing sequences based on priority levels
import heapq
class AirTrafficControl:
    """
    Manages aircraft landing sequences based on priority levels.
    Priority 1: Emergency (Critical)
    Priority 2: Low Fuel (Urgent)
    Priority 3: Scheduled (Normal)
    """
    def __init__(self):
        # Min-heap stores tuples: (priority_level, flight_id)
        self.landing_queue = []
    def request_landing(self, flight_id, priority):
        """
        Adds a flight to the landing sequence with a specific priority.
        :param flight_id: Unique identifier for the aircraft (e.g., 'DAL452').
        :param priority: Integer priority level (1=Highest, 3=Lowest).
        """
        heapq.heappush(self.landing_queue, (priority, flight_id))
        status = {1: "EMERGENCY", 2: "LOW FUEL", 3: "SCHEDULED"}.get(priority, "NORMAL")
        print(f"Inbound: {flight_id} | Status: {status} | Priority Level: {priority}")
    def clear_for_landing(self):
        """Clears the highest-priority aircraft for landing."""
        if not self.landing_queue:
            print("No aircraft currently in the landing pattern.")
            return None

        priority, flight_id = heapq.heappop(self.landing_queue)
        print(f"\n>>> ATC CLEARANCE: {flight_id}, you are cleared for landing. Runway 09R.")
        return flight_id

    def view_landing_pattern(self):
        """Displays all aircraft currently waiting to land, sorted by priority."""
        if not self.landing_queue:
            print("Landing pattern is clear.")
        else:
            # We sort a copy to display without altering the heap structure
            sorted_pattern = sorted(self.landing_queue)
            print("\n--- Current Landing Sequence ---")
            for p, f_id in sorted_pattern:
                status = {1: "EMERGENCY", 2: "LOW FUEL", 3: "SCHEDULED"}.get(p, "NORMAL")
                print(f"[{status}] Flight: {f_id}")
            print("--------------------------------\n")

# Example usage
if __name__ == "__main__":
    atc = AirTrafficControl()

    # Normal traffic arriving
    atc.request_landing("UAE202", 3)
    atc.request_landing("BAW117", 3)

    # An emergency situation arises
    atc.request_landing("AFR088", 1)  # Medical Emergency
    atc.request_landing("DLH400", 2)  # Low Fuel Warning

    # View the landing sequence (AFR088 should be at the top)
    atc.view_landing_pattern()

    # Clearing flights one by one
    atc.clear_for_landing() # Clears AFR088
    atc.clear_for_landing() # Clears DLH400
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/airTrafficControl.py
Inbound: UAE202 | Status: SCHEDULED | Priority Level: 3
Inbound: BAW117 | Status: SCHEDULED | Priority Level: 3
Inbound: AFR088 | Status: EMERGENCY | Priority Level: 1
Inbound: DLH400 | Status: LOW FUEL | Priority Level: 2

--- Current Landing Sequence ---
[EMERGENCY] Flight: AFR088
[LOW FUEL] Flight: DLH400
[SCHEDULED] Flight: BAW117
[SCHEDULED] Flight: UAE202
--------------------------------


>>> ATC CLEARANCE: AFR088, you are cleared for landing. Runway 09R.

>>> ATC CLEARANCE: DLH400, you are cleared for landing. Runway 09R.
```

Code 2:

```
C: > Users > PC > Downloads > aiac > emergencyLanding.py > ...
#write a python program to print Emergency Landing Priority – Aircraft in emergency get priority
class EmergencyLandingPriority:
    def __init__(self):
        self.emergency_flights = []

    def add_emergency_flight(self, flight_id):
        self.emergency_flights.append(flight_id)

    def get_priority_flights(self):
        return self.emergency_flights
    def display_priority_flights(self):
        print("Emergency Landing Priority – Aircraft in emergency get priority")
        print("------------------------------")
        for f_id in self.emergency_flights:
            status = "Emergency"
            print(f"Flight ID: {f_id}, Status: {status}")
        print("------------------------------\n")
        def clear_emergency_flights(self):
            self.emergency_flights.clear()
            def __str__(self):
                return f"EmergencyLandingPriority with {len(self.emergency_flights)} emergency flights."
            # Example usage
if __name__ == "__main__":
    elp = EmergencyLandingPriority()
    elp.add_emergency_flight("FL123")
    elp.add_emergency_flight("FL456")
    elp.display_priority_flights()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/emergencyLanding.py
Emergency Landing Priority – Aircraft in emergency get priority
------------------------------
Flight ID: FL123, Status: Emergency
Flight ID: FL456, Status: Emergency
------------------------------
```

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

• Select appropriate data structures from the list.
• Justify your choices clearly.
• Implement one selected feature as a Python program using AI- assisted code generation.

Expected Output:

• Table mapping features to data structures with justification.
• Working Python code with docstrings and comments.

Feature	Data Structure
1. News Feed	Linked List
2. Trending Hashtags	Priority Queue
3. User Profile Search	Hash Table
4. Friend Network	Graph
5. Message Undo	Stack

Code 1:

C: > Users > PC > Downloads > aiac > trendingTracker.py > ...

```python
#write a python program that simulates a social media platform's trending topics feature.
import heapq
class TrendingTracker:
    """
    Tracks hashtag frequency and retrieves top trending topics
    using a Priority Queue (Max-Heap simulation).
    """
    def __init__(self):
        # Dictionary to keep track of hashtag counts: {hashtag: count}
        self.hashtag_counts = {}

    def add_tag(self, hashtag):
        """Increments the usage count of a specific hashtag."""
        hashtag = hashtag.lower().strip("#")
        self.hashtag_counts[hashtag] = self.hashtag_counts.get(hashtag, 0) + 1
        print(f"Update: #{hashtag} is being talked about!")

    def get_trending(self, n=3):
        """
        Retrieves the top N trending hashtags.
        Uses a heap to find the largest elements efficiently.
        """
        if not self.hashtag_counts:
            print("No hashtags found.")
            return []

        # Convert dictionary items to a list of tuples (count, tag)
        # We use heapq.nlargest to get the top N items in O(D log N) time
        # where D is the number of unique hashtags.
        trending = heapq.nlargest(n, self.hashtag_counts.items(), key=lambda item: item[1])

        print(f"\n--- Top {n} Trending Hashtags ---")
        for rank, (tag, count) in enumerate(trending, 1):
            print(f"{rank}. #{tag} ({count} posts)")
        print("----------------------------------")
        return trending

# --- Social Media Simulation ---
if __name__ == "__main__":
    social_app = TrendingTracker()

    # Simulating users posting with hashtags
    social_app.add_tag("Python")
    social_app.add_tag("AI")
    social_app.add_tag("Coding")
    social_app.add_tag("AI")
    social_app.add_tag("Tech2026")
    social_app.add_tag("AI")
    social_app.add_tag("Python")

    # Retrieve the top trending tags
    social_app.get_trending(2)
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/trendingTracker.py
Update: #python is being talked about!
Update: #ai is being talked about!
Update: #coding is being talked about!
Update: #ai is being talked about!
Update: #tech2026 is being talked about!
Update: #ai is being talked about!
Update: #python is being talked about!

--- Top 2 Trending Hashtags ---
1. #ai (3 posts)
2. #python (2 posts)
----------------------------------
```

Code 2:

C: > Users > PC > Downloads > aiac > socialGraph.py > ...

```python
# write a python program that simulates a social media platform using Graphs
class SocialGraph:
    """
    Represents a social media network where users are nodes
    and friendships are undirected edges in a Graph.
    """
    def __init__(self):
        # Adjacency List: { 'User': set(['Friend1', 'Friend2']) }
        self.network = {}

    def add_user(self, username):
        """Adds a new user to the social platform."""
        if username not in self.network:
            self.network[username] = set()
            print(f"User @{username} joined the platform.")

    def add_friendship(self, user1, user2):
        """Creates a bidirectional friendship between two users."""
        if user1 in self.network and user2 in self.network:
            self.network[user1].add(user2)
            self.network[user2].add(user1)
            print(f"@{user1} and @{user2} are now friends!")
        else:
            print("Error: One or both users do not exist.")

    def get_mutual_friends(self, user1, user2):
        """Finds common connections between two users using set intersection."""
        if user1 in self.network and user2 in self.network:
            mutuals = self.network[user1].intersection(self.network[user2])
            print(f"\nMutual Friends between @{user1} and @{user2}: {list(mutuals)}")
            return mutuals
        return set()

    def show_network(self):
        """Displays the entire friend list for all users."""
        print("\n--- Social Media Network Graph ---")
        for user, friends in self.network.items():
            print(f"@{user} -> {', '.join(friends) if friends else 'No friends yet'}")
        print("----------------------------------")

# --- Simulation ---
if __name__ == "__main__":
    social_app = SocialGraph()

    # Onboarding users
    users = ["Alice", "Bob", "Charlie", "David"]
    for u in users:
        social_app.add_user(u)

    # Creating connections
    social_app.add_friendship("Alice", "Bob")
    social_app.add_friendship("Alice", "Charlie")
    social_app.add_friendship("Bob", "David")
    social_app.add_friendship("Charlie", "David")

    # Display the graph
    social_app.show_network()

    # Find mutual friends
    social_app.get_mutual_friends("Alice", "David")
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/socialGraph.py
User @Alice joined the platform.
User @Bob joined the platform.
User @Charlie joined the platform.
User @David joined the platform.
@Alice and @Bob are now friends!
@Alice and @Charlie are now friends!
@Bob and @David are now friends!
@Charlie and @David are now friends!

--- Social Media Network Graph ---
@Alice -> Bob, Charlie
@Bob -> David, Alice
@Charlie -> David, Alice
@David -> Bob, Charlie
----------------------------------

Mutual Friends between @Alice and @David: ['Bob', 'Charlie']
```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	**Data Structure**
1. Room Allotment	**Queue**
2. Medical Emergency	**Priority Queue**
3. Student Records	**Hash Table**
4. Block Connectivity	**Graph**
5. Visitor Entry Log	**Linked List**

Code 1:

C: > Users > PC > Downloads > aiac > hotelRegistry.py > HostelRegistry

```python
# write a python program to manage hostel student records using a Hash Table
class HostelRegistry:
    """
    A system to manage hostel student records using a Hash Table.
    Provides O(1) average time complexity for rapid data retrieval.
    """
    def __init__(self):
        # Internal dictionary to store student data
        # Key: Student ID (String), Value: Dictionary of Student Details
        self.records = {}

    def register_student(self, student_id, name, room_no, contact):
        """
        Adds a new student record to the hostel database.
        """
        if student_id in self.records:
            print(f"Update: Student ID {student_id} already exists. Updating details.")

        self.records[student_id] = {
            "name": name,
            "room_no": room_no,
            "contact": contact
        }
        print(f"Successfully registered: {name} in Room {room_no}")

    def get_student_info(self, student_id):
        """
        Retrieves student details instantly using their unique ID.
        """
        student = self.records.get(student_id)
        if student:
            print(f"\n--- Hostel Record: {student_id} ---")
            print(f"Name:    {student['name']}")
            print(f"Room:    {student['room_no']}")
            print(f"Contact: {student['contact']}")
            print("-" * 30)
```

```python
            print("-" * 30)
            return student
        else:
            print(f"\nError: Student ID {student_id} not found in the registry.")
            return None

    def checkout_student(self, student_id):
        """Removes a student from the registry upon hostel checkout."""
        if student_id in self.records:
            removed = self.records.pop(student_id)
            print(f"Student {removed['name']} has checked out and record is cleared.")
        else:
            print("Checkout failed: Student ID not found.")

# --- Hostel Management Simulation ---
if __name__ == "__main__":
    hostel_admin = HostelRegistry()

    # Registering students
    hostel_admin.register_student("STU001", "Aaryan Khan", "B-101", "+91-98765-43210")
    hostel_admin.register_student("STU002", "Priya Sharma", "A-204", "+91-87654-32109")
    hostel_admin.register_student("STU003", "Liam Smith", "C-305", "+91-76543-21098")

    # Instant lookup by ID
    hostel_admin.get_student_info("STU002")

    # Searching for a non-existent ID
    hostel_admin.get_student_info("STU999")

    # Removing a record after graduation/checkout
    hostel_admin.checkout_student("STU003")
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/hotelRegistry.py
Successfully registered: Aaryan Khan in Room B-101
Successfully registered: Priya Sharma in Room A-204
Successfully registered: Liam Smith in Room C-305

--- Hostel Record: STU002 ---
Name:    Priya Sharma
Room:    A-204
Contact: +91-87654-32109
------------------------------

Error: Student ID STU999 not found in the registry.
Student Liam Smith has checked out and record is cleared.
```

Code 2:

C: > Users > PC > Downloads > aiac > hostelMedicalSystem.py > HostelMedicalSystem > dispatch_aid

```python
#write a python program to manage hostel medical emergencies using a priority queue.
import heapq
class HostelMedicalSystem:
    """
    A system to manage hostel medical emergencies using a Priority Queue.
    Priority 1: Critical (Immediate attention required)
    Priority 2: Serious (Needs attention within the hour)
    Priority 3: Minor (Stable condition)
    """
    def __init__(self):
        # The heap stores tuples: (priority_level, student_name, room_no)
        self.emergency_heap = []
    def report_emergency(self, name, room, priority):
        """
        Adds an emergency case to the queue.
        :param name: Student's name.
        :param room: Student's room number.
        :param priority: Integer (1=Highest, 3=Lowest).
        """
        heapq.heappush(self.emergency_heap, (priority, name, room))
        status = {1: "CRITICAL", 2: "SERIOUS", 3: "MINOR"}.get(priority, "ROUTINE")
        print(f"Reported: {name} (Room {room}) | Status: {status}")
    def dispatch_aid(self):
        """Processes the highest priority emergency first."""
        if not self.emergency_heap:
            print("No active medical emergencies. All students safe.")
            return None
        priority, name, room = heapq.heappop(self.emergency_heap)
        print(f"\n[DISPATCHING MEDICAL AID] -> Student: {name}, Room: {room}")
        print(f"Reason: Priority Level {priority} condition.")
        return name
    def view_emergency_list(self):
        """Displays all pending medical cases sorted by urgency."""
        if not self.emergency_heap:
            print("The emergency queue is currently empty.")
        else:
            # Sort a copy to display in priority order without emptying the heap
```

```python
            # Sort a copy to display in priority order without emptying the heap
            sorted_emergencies = sorted(self.emergency_heap)
            print("\n--- Current Emergency Dashboard ---")
            for p, name, room in sorted_emergencies:
                status = {1: "CRITICAL", 2: "SERIOUS", 3: "MINOR"}.get(p)
                print(f"[{status}] Student: {name} | Room: {room}")
            print("-----------------------------------\n")

# --- Simulation ---
if __name__ == "__main__":
    hostel_med = HostelMedicalSystem()

    # Reporting emergencies as they come in
    hostel_med.report_emergency("John Doe", "B-201", 3)   # Minor
    hostel_med.report_emergency("Jane Smith", "A-104", 1) # Critical
    hostel_med.report_emergency("Alex Wong", "C-305", 2)  # Serious

    # See the dashboard (Jane should be at the top)
    hostel_med.view_emergency_list()

    # Processing the most urgent case
    hostel_med.dispatch_aid()
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/hostelMedicalSystem.py
Reported: John Doe (Room B-201) | Status: MINOR
Reported: Jane Smith (Room A-104) | Status: CRITICAL
Reported: Alex Wong (Room C-305) | Status: SERIOUS

--- Current Emergency Dashboard ---
[CRITICAL] Student: Jane Smith | Room: A-104
[SERIOUS] Student: Alex Wong | Room: C-305
[MINOR] Student: John Doe | Room: B-201
-----------------------------------


[DISPATCHING MEDICAL AID] -> Student: Jane Smith, Room: A-104
Reason: Priority Level 1 condition.
```

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack
o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Feature	**Data Structure**
1. Complaint Handling	**Queue**
2. Fault Priority	**Priority Queue**
3. Consumer Lookup	**Hash Table**
4. Power Grid	**Graph**
5. Reading History	**Linked List**

Code 1:

C: > Users > PC > Downloads > aiac > gridFaultManager.py > ...

```python
#write a python program that manages electricity grid faults based on urgency and impact.
import heapq
class GridFaultManager:
    """
    Manages electricity grid faults based on urgency and impact.
    Priority 1: Emergency (Grid Collapse / Safety Hazard)
    Priority 2: High Impact (Whole Neighborhood Outage)
    Priority 3: Low Impact (Individual House Fault)
    """
    def __init__(self):
        # Min-heap stores tuples: (priority_level, fault_id, location)
        self.fault_queue = []

    def report_fault(self, fault_id, location, priority):
        """
        Adds a new fault to the management system.
        :param fault_id: Unique tracking ID for the fault.
        :param location: The area or substation affected.
        :param priority: Integer (1=Critical, 3=Minor).
        """
        heapq.heappush(self.fault_queue, (priority, fault_id, location))
        severity = {1: "CRITICAL", 2: "URGENT", 3: "ROUTINE"}.get(priority, "NORMAL")
        print(f"Logged: {fault_id} at {location} | Severity: {severity}")

    def dispatch_technician(self):
        """Dispatches the next technician to the highest priority fault."""
        if not self.fault_queue:
            print("No active faults. All systems normal.")
            return None

        priority, f_id, loc = heapq.heappop(self.fault_queue)
        print(f"\n[DISPATCH] Technician sent to {loc} for Fault ID: {f_id} (Priority {priority})")
        return f_id

    def view_active_faults(self):
        """Displays all pending faults sorted by their priority."""
        if not self.fault_queue:
            print("No pending faults.")
        else:
            # Sort a copy for display to keep heap integrity
            active_list = sorted(self.fault_queue)
            print("\n--- Active Grid Faults (Sorted by Priority) ---")
            for p, f_id, loc in active_list:
                print(f"Level {p} | ID: {f_id} | Location: {loc}")
            print("----------------------------------------------\n")
# Example usage
if __name__ == "__main__":
    smart_grid = GridFaultManager()

    # Reporting faults as they occur
    smart_grid.report_fault("F-101", "Baker Street", 3)
    smart_grid.report_fault("F-505", "City Hospital Substation", 1) # Highest Priority
    smart_grid.report_fault("F-202", "Industrial Zone", 2)

    # View the priority list
    smart_grid.view_active_faults()
    # Processing/Dispatching
    smart_grid.dispatch_technician() # Should dispatch to City Hospital
    smart_grid.dispatch_technician() # Should dispatch to Industrial Zone
```

Output 1:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/gridFaultManager.py
Logged: F-101 at Baker Street | Severity: ROUTINE
Logged: F-505 at City Hospital Substation | Severity: CRITICAL
Logged: F-202 at Industrial Zone | Severity: URGENT

--- Active Grid Faults (Sorted by Priority) ---
Level 1 | ID: F-505 | Location: City Hospital Substation
Level 2 | ID: F-202 | Location: Industrial Zone
Level 3 | ID: F-101 | Location: Baker Street
----------------------------------------------


[DISPATCH] Technician sent to City Hospital Substation for Fault ID: F-505 (Priority 1)

[DISPATCH] Technician sent to Industrial Zone for Fault ID: F-202 (Priority 2)
```

Code 2:

C: > Users > PC > Downloads > aiac > electyricityBoardRegistry.py > ...

```python
#write a python program that simulates an electricity board registry using a hash table for efficient consumer record management.
class ElectricityBoardRegistry:
    """
    A system to manage consumer records using a Hash Table.
    Enables O(1) average time complexity for account lookups.
    """
    def __init__(self):
        # Dictionary acts as the Hash Table
        # Key: Consumer ID, Value: Dictionary of account details
        self.consumer_db = {}

    def add_consumer(self, consumer_id, name, address, initial_balance=0.0):
        """Adds a new consumer record to the database."""
        if consumer_id in self.consumer_db:
            print(f"Update: Consumer ID {consumer_id} already exists.")
        else:
            self.consumer_db[consumer_id] = {
                "name": name,
                "address": address,
                "balance": initial_balance,
                "status": "Active"
            }
            print(f"Consumer {name} registered successfully with ID: {consumer_id}")

    def lookup_consumer(self, consumer_id):
        """Retrieves consumer information instantly using their unique ID."""
        record = self.consumer_db.get(consumer_id)
        if record:
            print(f"\n--- Consumer Record: {consumer_id} ---")
            print(f"Name:    {record['name']}")
            print(f"Address: {record['address']}")
            print(f"Balance: ${record['balance']:.2f}")
            print(f"Status:  {record['status']}")
            print("-" * 35)
            return record
        else:
            print(f"\nError: Consumer ID {consumer_id} not found.")
```

```python
            return None

    def update_balance(self, consumer_id, amount):
        """Updates the outstanding bill amount for a consumer."""
        if consumer_id in self.consumer_db:
            self.consumer_db[consumer_id]["balance"] += amount
            new_bal = self.consumer_db[consumer_id]["balance"]
            print(f"Balance updated for {consumer_id}. New Balance: ${new_bal:.2f}")
        else:
            print("Error: Cannot update balance. ID not found.")

# Example usage
if __name__ == "__main__":
    eb_system = ElectricityBoardRegistry()

    # Adding consumers
    eb_system.add_consumer("CN-9001", "Avery Brooks", "123 Solar Lane", 45.50)
    eb_system.add_consumer("CN-9002", "Jordan Lee", "456 Volt Blvd", 12.00)
    eb_system.add_consumer("CN-9003", "Sam Rivera", "789 Ohm Court", 102.30)

    # Instant lookup test
    eb_system.lookup_consumer("CN-9001")

    # Update balance (e.g., new monthly reading)
    eb_system.update_balance("CN-9001", 30.25)

    # Verification lookup
    eb_system.lookup_consumer("CN-9001")
```

Output 2:

```
PS E:\Microsoft VS Code> & C:\Users\PC\AppData\Local\Programs\Python\Python314\python.exe c:/Users/PC/Downloads/aiac/electyricityBoardRegistry.py
Consumer Avery Brooks registered successfully with ID: CN-9001
Consumer Jordan Lee registered successfully with ID: CN-9002
Consumer Sam Rivera registered successfully with ID: CN-9003

--- Consumer Record: CN-9001 ---
Name:    Avery Brooks
Address: 123 Solar Lane
Balance: $45.50
Status:  Active
----------------------------------
Balance updated for CN-9001. New Balance: $75.75

--- Consumer Record: CN-9001 ---
Name:    Avery Brooks
Address: 123 Solar Lane
Balance: $75.75
Status:  Active
----------------------------------
```